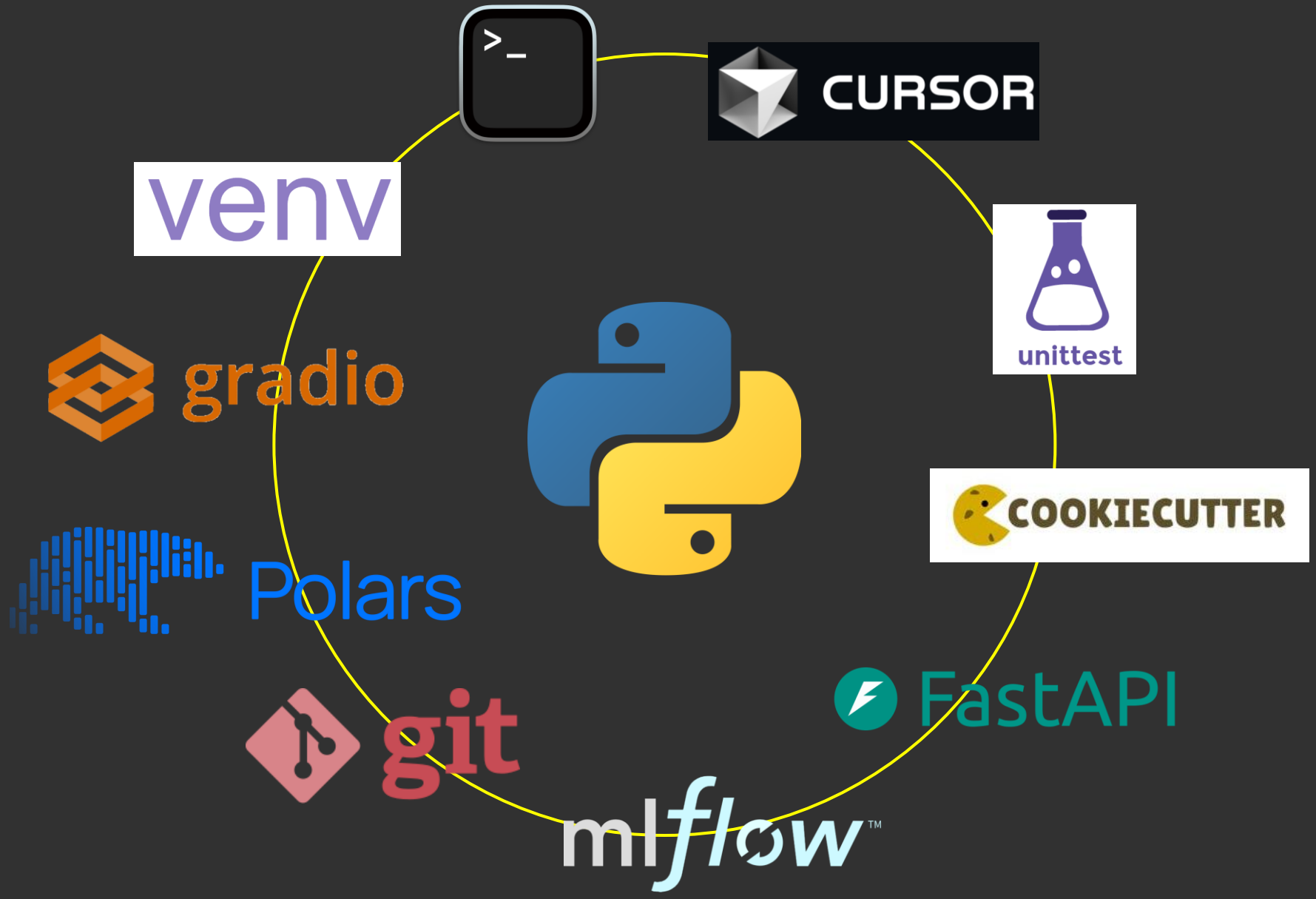
DAPT 619

Vishal Patel

Fall 2025


>_
CURSOR
venv
unittest
gradio
COOKIECUTTER
Polars
FastAPI
git
mlflow

Gradio

gradio

Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild

Abubakar Abid [*1 2] Ali Abdalla [*2] Ali Abid [*2] Dawood Khan [*2] Abdulrahman Alfozan [2] James Zou [3]

6 Jun 2019

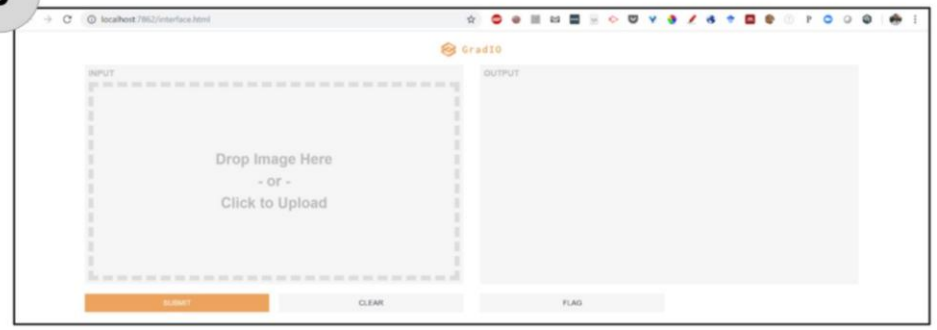
The machine learning researcher defines the input and output interface types, and launches the interface either inline or in a new browser tab.

The interfaces launches, and optionally, a public link is created that allows remote collaborators to input their own data into the model.

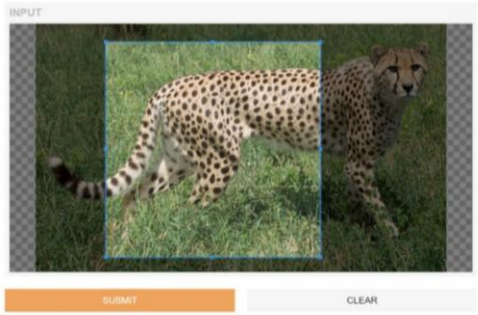
a **Introducing Gradio**

```
import tensorflow as tf
import gradio

mdl = tf.keras.models.Sequential()
# ... define and train the model as you would normally

io = gradio.Interface(inputs="imageupload",
    outputs="label", model_type="keras", model=mdl)
io.launch()
```

b



c



d

The users of the interface can also manipulate the model in natural ways, such as cropping images or obscuring parts of the image.

All model computation is done by the host. The user can interact with the model on their browser without local computation and can provide real-time feedback which is sent to the host.

The **Interface** class is designed to create demos for machine learning models which accept one or more inputs and return one or more outputs.

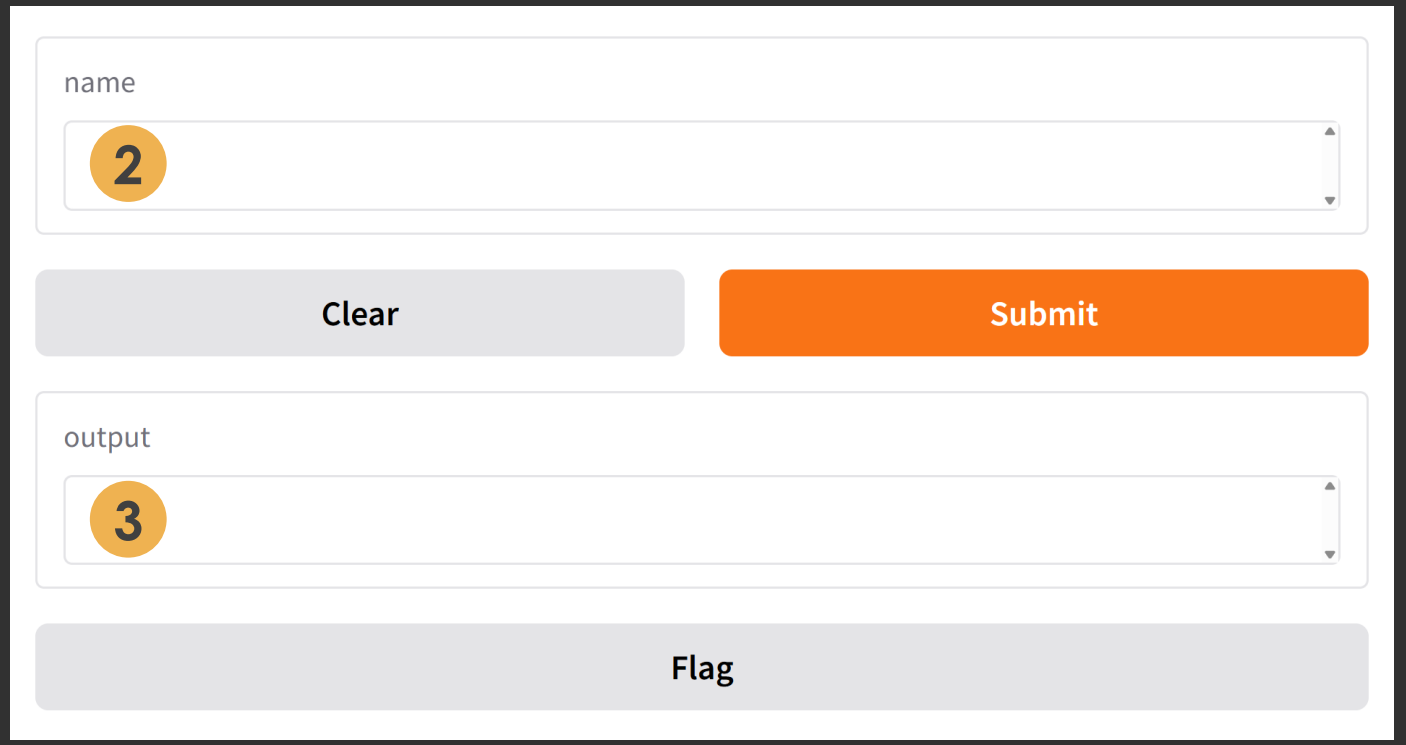
1. **fn**: the function to wrap a user interface (UI) around

Components

2. **inputs**: the Gradio component(s) to use for the input. The number of components should match the number of arguments in your function.

3. **outputs**: the Gradio component(s) to use for the output. The number of components should match the number of return values from your function.

```python
import gradio as gr

def greet(name):
    return "Hello " + name + "!"

demo = gr.Interface(fn=greet, inputs="textbox", outputs="textbox")

demo.launch()
```



Gradio

```
pip install gradio
```

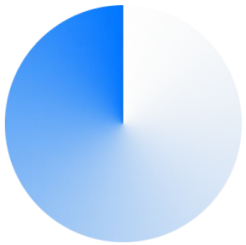
Polars

Polars

DataFrames for the new era

250M+ Downloads to date 35k+ Github stars

BENEFITS



01

Fast

Polars is written from the ground up with performance in mind. Its multi-threaded query engine is written in Rust and designed for effective parallelism. Its vectorized and columnar processing enables cache-coherent algorithms and high performance on modern processors.

02

Easy to use

You will feel right at home with Polars if you are familiar with data wrangling. Its expressions are intuitive and empower you to write code which is readable and performant at the same time.

03

Open source

Polars is and always will be open source. Driven by an active community of developers, everyone is encouraged to add new features and contribute. Polars is free to use under the MIT license.

Polars: Philosophy

The goal of Polars is to provide a lightning fast DataFrame library that:

- Utilizes all available cores on your machine.
- Optimizes queries to reduce unneeded work/memory allocations.
- Handles datasets much larger than your available RAM.
- A consistent and predictable API.
- Adheres to a strict schema.

A Vectorized Query Engine

Polars is a vectorized query engine, which means it processes data in chunks (columns or "vectors“) at a time, rather than looping through rows one by one.

So when you write something like:

```
df.with_columns((pl.col("price") * 1.1).alias("price_with_tax"))
```

Polars doesn’t touch one row at a time; it performs the multiplication across the entire column in a single, efficient operation.

Polars

```
pip install polars
```

01_polars_wrangle_tutorial.ipynb

02_lazy_evaluation.ipynb

Project Code Structure

Noble's Principles

1. Someone unfamiliar with your project should be able to look at your files and understand in detail what you did and why.

2. Everything you do, you will probably have to do over again.

A Quick Guide to Organizing Computational Biology Projects:
https://dx.plos.org/10.1371/journal.pcbi.1000424

“Let us change our traditional attitude to
the construction of programs.

Instead of imagining that our main task is
to instruct a computer what to do,
let us concentrate rather on explaining to human beings
what we want a computer to do.”

– Donald E. Knuth

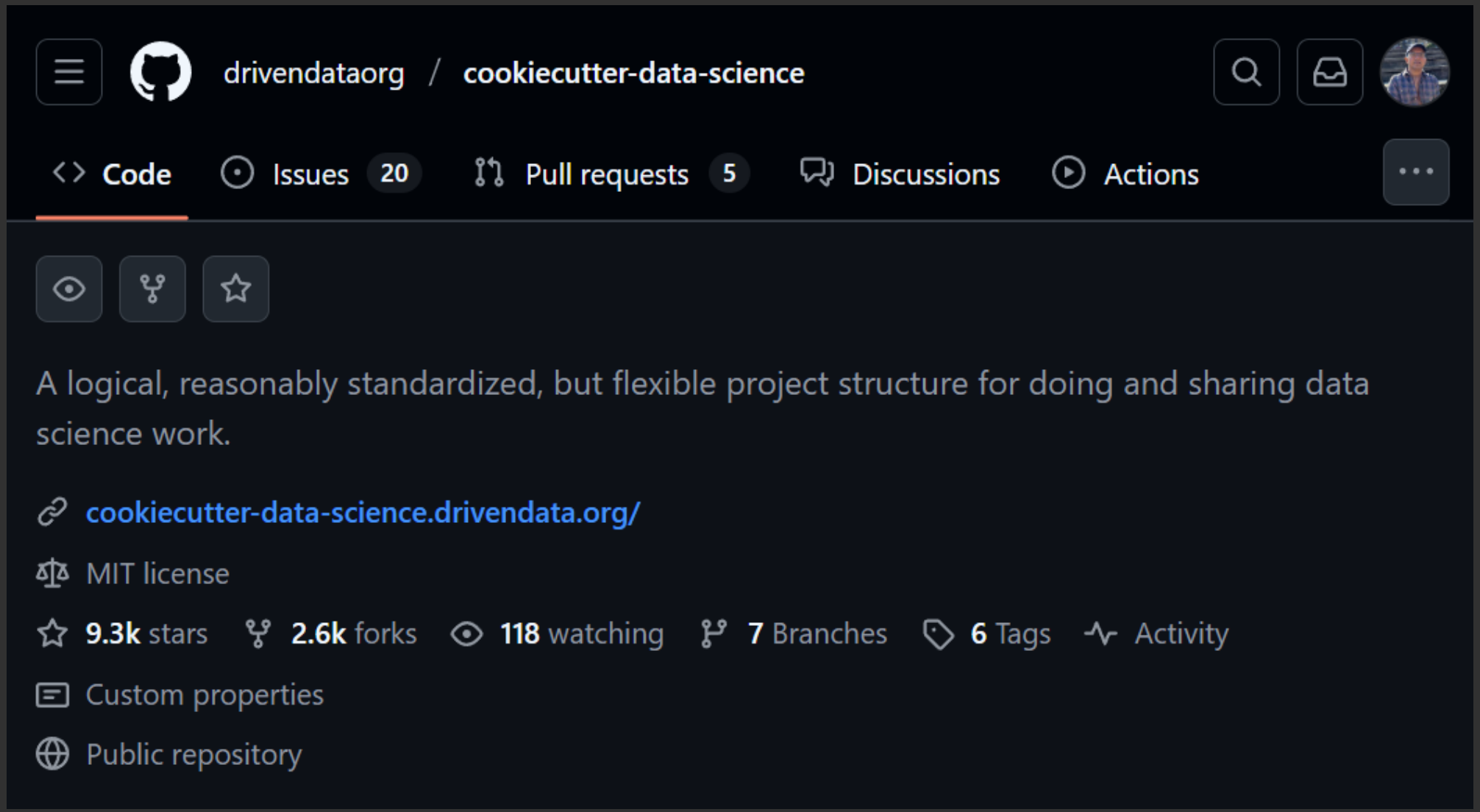
Elements of a Data Science Project

README.md

Why this project exists, how things are organized, conventions used in the project, etc.

Data	Models	Notebooks	Reports	Scripts	Tests
Raw, Interim, Processed	Model artifacts	Jupyter Notebooks	Plots, Excel files,...	`.py` files	Unit tests

Configs	Secrets	Requirements
Paths, options, ...	API keys, ...	Python dependencies


drivendataorg / cookiecutter-data-science
Code
Issues 20
Pull requests 5
Discussions
Actions
A logical, reasonably standardized, but flexible project structure for doing and sharing data science work.
cookiecutter-data-science.drivendata.org/
MIT license
9.3k stars
2.6k forks
118 watching
7 Branches
6 Tags
Activity
Custom properties
Public repository

Cookiecutter Data Science: Opinions

1 Data analysis is a directed acyclic graph.

The best way to ensure reproducibility is to treat your data analysis pipeline as a directed acyclic graph (DAG).

This means each step of your analysis is a node in a directed graph with no loops. You can run through the graph forwards to recreate any analysis output, or you can trace backwards from an output to examine the combination of code and data that created it.

Cookiecutter Data Science: Opinions

2 Raw data is immutable.

That proper data analysis is a DAG means that raw data must be treated as immutable—it's okay to read and copy raw data to manipulate it into new outputs, but never okay to change it in place.

Cookiecutter Data Science: Opinions

3 Data should (mostly) not be kept in source control.

Another consequence of treating data as immutable is that data doesn't need source control in the same way that code does.

Therefore, by default, the `data/` folder is included in the `.gitignore` file.

If you have a small amount of data that rarely changes, you may want to include the data in the repository.

Cookiecutter Data Science: Opinions

4

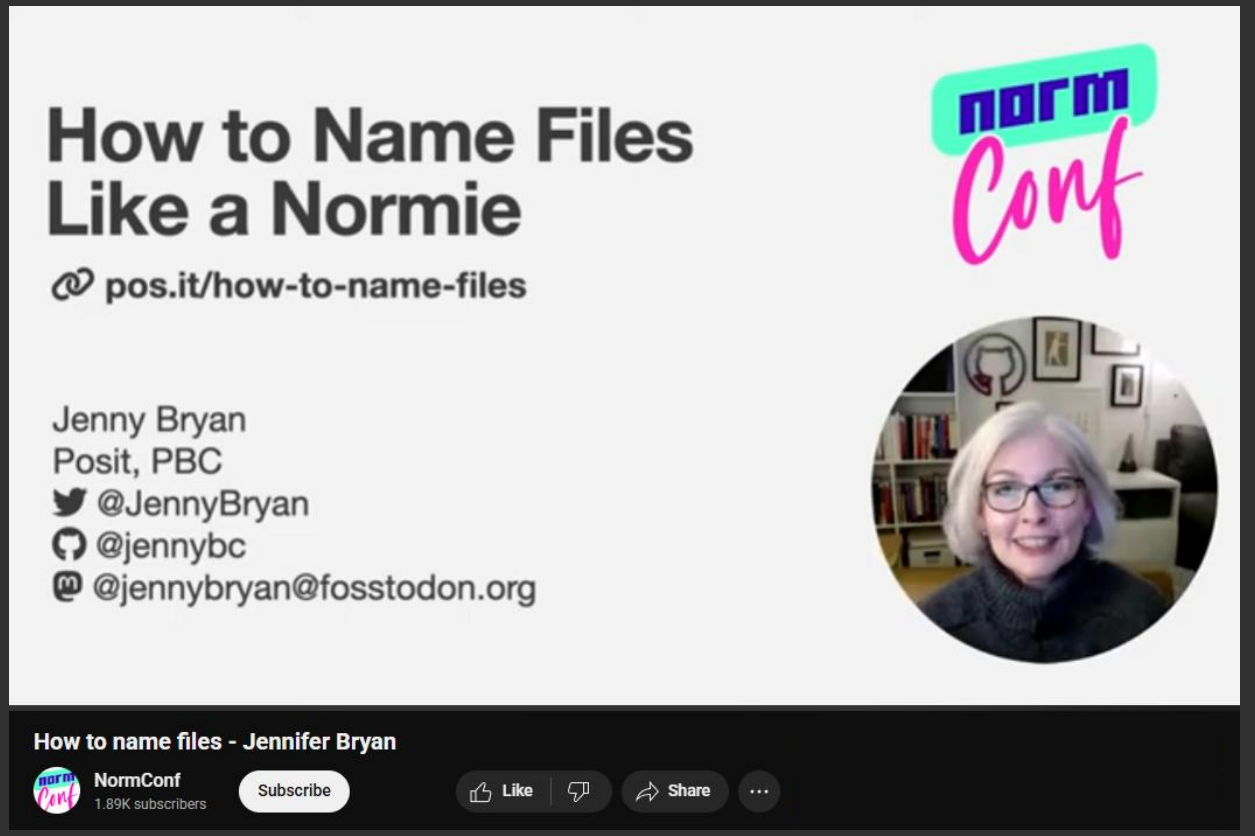
Notebooks are for exploration and communication, source files are for repetition.

Source code is superior for replicability because it is more portable, can be tested more easily, and is easier to code review.

https://cookiecutter-data-science.drivendata.org/opinions/

Cookiecutter

How to Name Files



1. ProjectBrief_FINAL (2).pptx
2. Sales Report 12-04-09.csv
3. 20250924_lab_notes.md
4. 2025-09-24_project-brief_v03.pptx
5. 2012-04-09_sales-report.csv